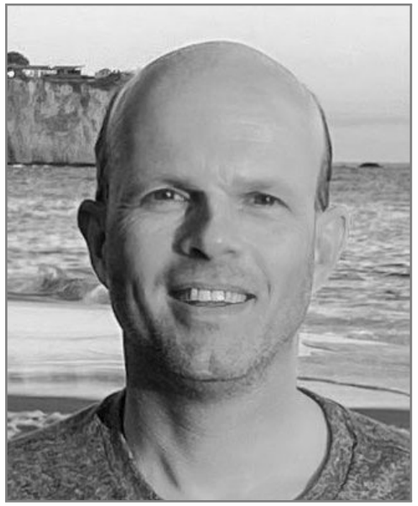
Releasing

Yegor Bugayenko

Lecture #7 out of 8
80 minutes

The slidedeck was presented by the author in this YouTube Video.

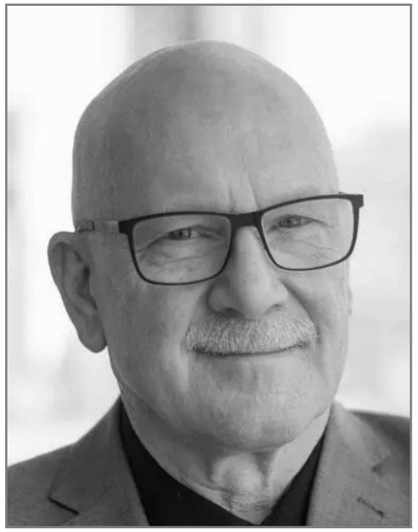
All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.



Andre Van Der Hoek

"Simply making available and retrieving interdependent components individually neither facilitates independent software development nor encourages widespread use of large systems of systems."

— Andre Van Der Hoek et al. "Software Release Management". In: *ACM SIGSOFT Software Engineering Notes* 22.6 (1997), pp. 159–175. DOI: 10.1145/267896.267909



Günther Ruhe

"Release planning is typically done *ad hoc* and not based on sound models and methodologies. This is even the case when planning involves several hundred features."

— Günther Ruhe and Moshood Omolade Saliu. "The Art and Science of Software Release Planning". In: *IEEE Software* 22.6 (2005), pp. 47–53. DOI: 10.1109/MS.2005.164

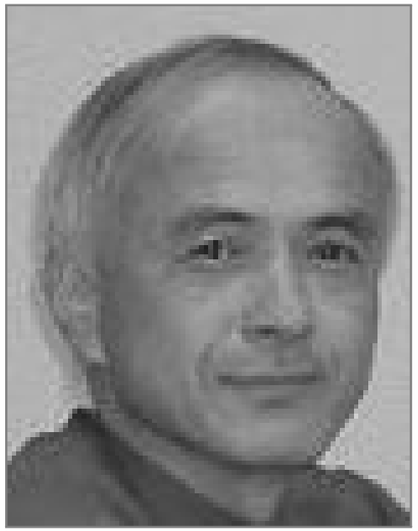
1. Release when it's different, not when it's good.



Kazu Okumoto

"**Goel-Okumoto model**: An important problem of practical concern is the determination of the point when testing should stop and the system can be considered ready for release, that is, the determination of the software release time. Two criteria are investigated: software reliability and total expected cost."

— Kazu Okumoto and Amrit L. Goel. "Optimum Release Time for Software Systems Based on Reliability and Cost Criteria". In: *Journal of Systems and Software* 1.1 (1979), pp. 315–318. DOI: 10.1016/0164-1212(79)90033-5



Shigeru Yamada

"This paper extends the problem of Okumoto and Goel [8] by evaluating both criteria simultaneously. We discuss optimal software release policies which minimize a total average software cost under the constraint of satisfying a software reliability requirement."

— Shigeru Yamada and Shunji Osaki. "Cost-Reliability Optimal Release Policies for Software Systems". In: *IEEE Transactions on Reliability* 34.5 (1985), pp. 422–424. DOI: 10.1109/TR.1985.5222222

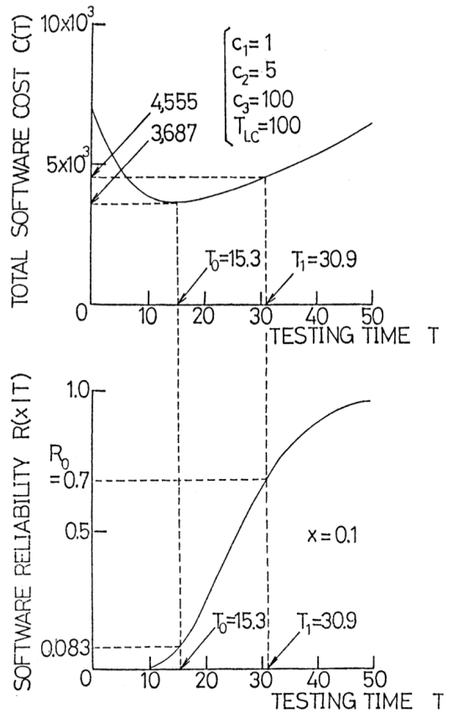
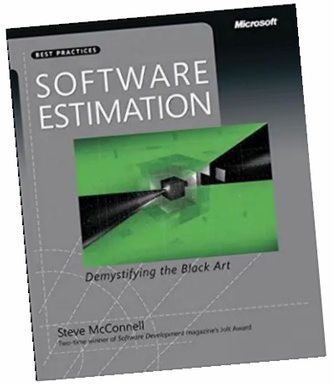


Fig. 1. An illustration of the cost-reliability optimal release problem.

"The optimum software release time is the testing time which comes closest to satisfying some pre-specified software reliability."

Source: Shigeru Yamada and Shunji Osaki. "Cost-Reliability Optimal Release Policies for Software Systems". In: *IEEE Transactions on Reliability* 34.5 (1985), pp. 422–424. DOI: 10.1109/TR.1985.5222222

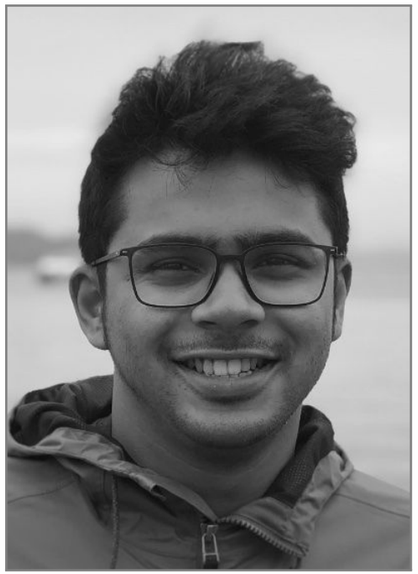




Steve McConnell

"The question of whether to release software is a treacherous one. The answer must teeter on the line between releasing poor quality software early and releasing high quality software late. The questions of 'Is the software good enough to release now?' and 'When will the software be good enough to release?' can become critical to a company's survival."

— Steve McConnell. *Software Project Survival Guide*. Microsoft Press, 1998. DOI: 10.5555/270015



Nasif Imtiaz

"We find that the open source packages are typically fast in releasing security fixes, as the median release comes within 4 days of the corresponding security fix. However, 25% of the releases still have a delay of at least 20 days."

— Nasif Imtiaz, Aniqa Khanom, and Laurie Williams. "Open or Sneaky? Fast or Slow? Light or Heavy? Investigating Security Releases of Open Source Packages". In: *IEEE Transactions on Software Engineering* 49.4 (2022), pp. 1540–1560. DOI: 10.1109/TSE.2022.3181010

2. Fully automate the release process, avoiding any manual intervention.

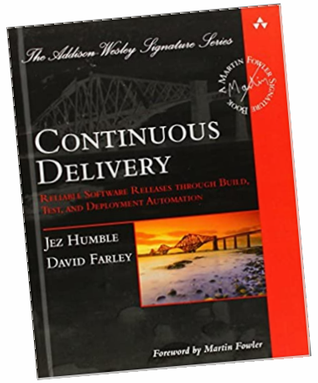




Michael Nygard

"Releases should be about as big an event as getting a haircut. There's an added benefit of frequent releases: it forces you to get really good at doing releases and deployments."

— Michael Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007. DOI: 10.5555/1200767





Jez Humble

"Over time, deployments should tend towards being fully automated. There should be two tasks for a human being to perform to deploy software into a development, test, or production environment: to pick the version and environment and to press the 'deploy' button."

— Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Pearson Education, 2010. DOI: 10.5555/1869904

3. Release frequently.

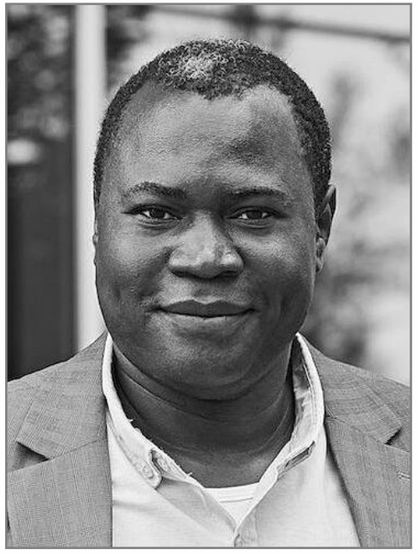
Respectfully disagree!



Victor Basili

"If improving productivity is the main concern, then it may be wise to try to avoid scheduling small error correction releases. Instead the manager should try, when possible, to package small error corrections in a release with larger enhancements. "

— Victor Basili et al. "Understanding and Predicting the Process of Software Maintenance Releases". In: *Proceedings of the 18th International Conference on Software Engineering*. IEEE. 1996, pp. 464–474. DOI: 10.1109/ICSE.1996.493441

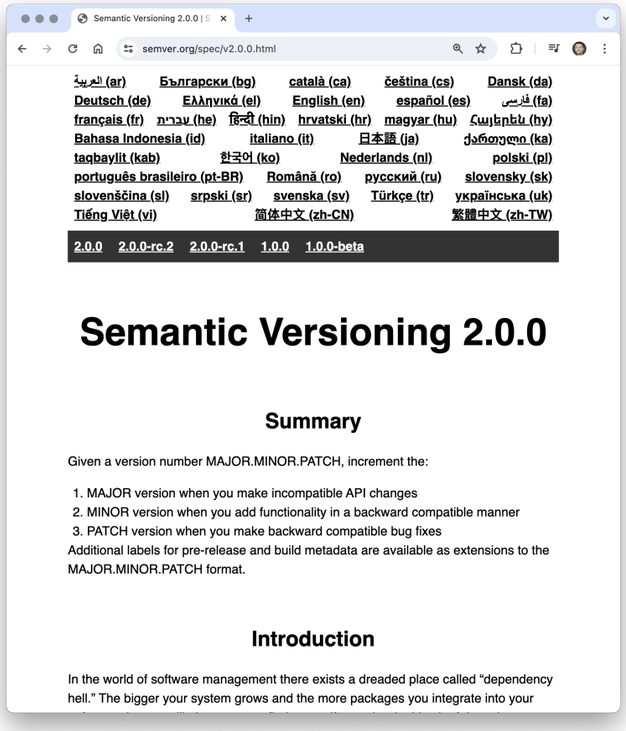


Foutse Khomh

"We found that (1) with shorter release cycles, users do not experience significantly more post-release bugs and (2) bugs are fixed faster, yet (3) users experience these bugs earlier during software execution (the program crashes earlier)."

— Foutse Khomh et al. "Do Faster Releases Improve Software Quality? An Empirical Case Study of Mozilla Firefox". In: *Proceedings of the 9th Working Conference on Mining Software Repositories (MSR)*. IEEE. 2012, pp. 179–188. DOI: 10.1109/MSR.2012.6224279

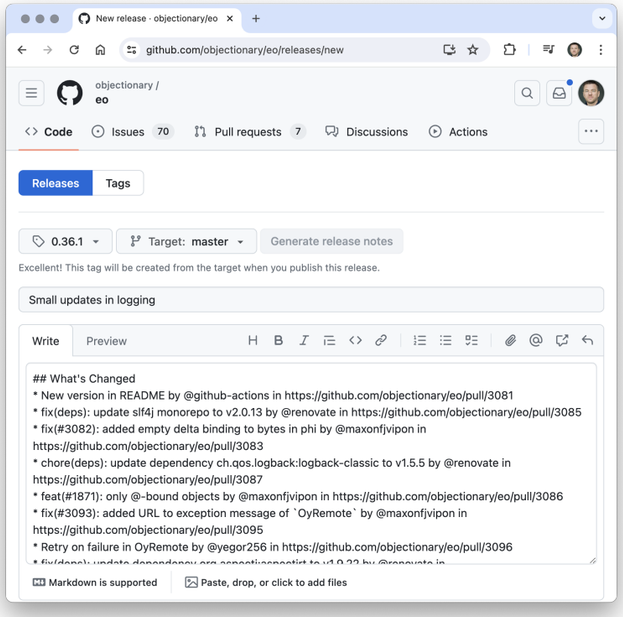
4. Use SemVer.



“The implication of semantic versioning is that clients may rely on dependencies subject to flexible version constraints, like `1.2.*`. Such a client may safely upgrade to new micro versions (e.g., from `1.2.3` to `1.2.4`), fully automated.”

Source: Patrick Lam, Jens Dietrich, and David J. Pearce. “Putting the Semantics into Semantic Versioning”. In: *Proceedings of the SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 2020, pp. 157–179. DOI: 10.1145/3426428.3426922

5. Generate release notes automatically.



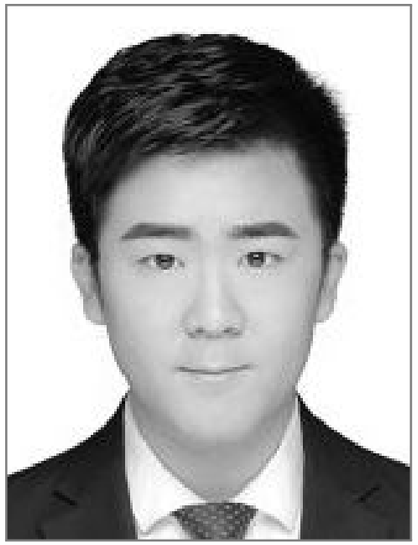
"Automatically generated release notes provide an automated alternative to manually writing release notes for your GitHub releases. With automatically generated release notes, you can quickly generate an overview of the contents of a release."

`https://docs.github.com/en/repositories/releasing-projects-on-github/automatically-generated-release-notes`

Some Release Notes Generators:

- ClickUp
- Taskade
- Zeda
- Aha
- ReleasesNotes
- ScribeHow
- Released
- ai-github-release-notes

Try Google search with "generate release notes with AI"



Jianyu Wu

"We find that: 1) RN producers are more likely to miss information than to include incorrect information, especially for breaking changes; 2) improper layout may bury important information and confuse users; 3) many users find RNs inaccessible due to link deterioration, lack of notification, and obfuscate RN locations; 4) automating and regulating RN production remains challenging despite the great needs of RN producers."

— Jianyu Wu et al. "Demystifying Software Release Note Issues on GitHub". In: *Proceedings of the 30th International Conference on Program Comprehension*. 2022, pp. 602–613. DOI: 10.1145/3524610.3527919

6. Publish binaries.

Some Artifact Publishing Platforms:

- Maven Central for Java, Kotlin, Scala, Groovy, etc.
- Npm for JavaScript
- PyPi for Python
- RubyGems for Ruby
- Crates for Rus

References

[1] Victor Basili et al. "Understanding and Predicting the Process of Software Maintenance Releases". In: *Proceedings of the 18th International Conference on Software Engineering*. IEEE. 1996, pp. 464–474. DOI: 10.1109/ICSE.1996.493441.

[2] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*. Pearson Education, 2010. DOI: 10.5555/1869904.

[3] Nasif Imtiaz, Aniqa Khanom, and Laurie Williams. "Open or Sneaky? Fast or Slow? Light or Heavy? Investigating Security Releases of Open Source Packages". In: *IEEE Transactions on Software Engineering* 49.4 (2022), pp. 1540–1560. DOI: 10.1109/TSE.2022.3181010.

[4] Foutse Khomh et al. "Do Faster Releases Improve Software Quality? An Empirical Case Study of Mozilla Firefox". In: *Proceedings of the 9th Working Conference on Mining Software Repositories (MSR)*. IEEE. 2012, pp. 179–188. DOI: 10.1109/MSR.2012.6224279.

[5] Patrick Lam, Jens Dietrich, and David J. Pearce. "Putting the Semantics into Semantic Versioning". In: *Proceedings of the SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 2020, pp. 157–179. DOI: 10.1145/3426428.3426922.

[6] Steve McConnell. *Software Project Survival Guide*. Microsoft Press, 1998. DOI: 10.5555/270015.

[7] Michael Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2007. DOI: 10.5555/1200767.

[8] Kazu Okumoto and Amrit L. Goel. "Optimum Release Time for Software Systems Based on Reliability and Cost Criteria". In: *Journal of Systems and Software* 1.1 (1979), pp. 315–318. DOI: 10.1016/0164-1212(79)90033-5.

[9] Günther Ruhe and Moshood Omolade Saliu. "The Art and Science of Software Release Planning". In: *IEEE Software* 22.6 (2005), pp. 47–53. DOI: 10.1109/MS.2005.164.

[10] Andre Van Der Hoek et al. "Software Release Management". In: *ACM SIGSOFT Software Engineering Notes* 22.6 (1997), pp. 159–175. DOI: 10.1145/267896.267909.

[11] Jianyu Wu et al. "Demystifying Software Release Note Issues on GitHub". In: *Proceedings of the 30th International Conference on Program Comprehension*. 2022, pp. 602–613. DOI: 10.1145/3524610.3527919.

[12] Shigeru Yamada and Shunji Osaki. "Cost-Reliability Optimal Release Policies for Software Systems". In: *IEEE Transactions on Reliability* 34.5 (1985), pp. 422–424. DOI: 10.1109/TR.1985.5222222.